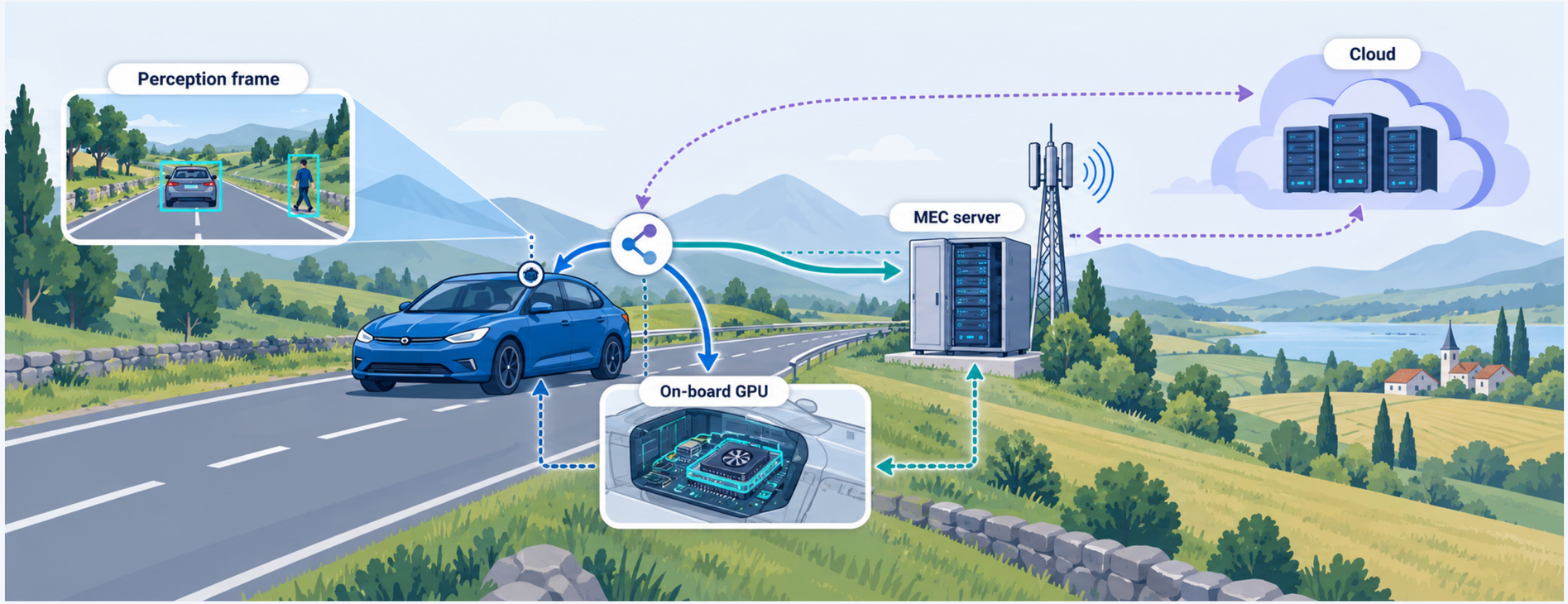
One perception frame. Three places to run the AI.

Camera frame → vehicle state + network prediction → QoS-aware choice → detection result back to the vehicle



ON-BOARD GPU

Local execution when remote tiers cannot meet the limits.

MEC SERVER

Prefer the nearby edge when its accuracy is sufficient.

CLOUD

Use remote capacity when feasible and appropriate.

Where should a vehicle run its perception AI?

The same camera task can run in the car, at a nearby edge server, or in the cloud.

A moving vehicle must keep detecting road users while its connection changes.

A small on-board model responds quickly. Larger remote models offer better detection accuracy, but add network delay.

The decision is repeated for every frame

Balance response time, model accuracy and server load. Keep a local fallback when remote execution fails.

A / B — **IN THE VEHICLE**

YOLOv8n • NPU / GPU • 8 / 5 ms inference

C — **NEARBY MEC SERVER**

YOLOv8m • 6 ms inference + network

D — **CLOUD DATA CENTER**

YOLOv8l • 4 ms inference + network

Demo values are configured simulation inputs.

From radio physics to a live placement decision

PhySDV connects synthetic data generation, learned network prediction and an explainable policy.

01
GENERATE

50,000 labeled samples
Five road environments
Radio + traffic + server load

02
PREDICT

Is the MEC reachable?
What network delay is expected?
How busy are the servers?

03
SELECT

Update the four execution tiers
Check the QoS limits
Choose a tier and explain why

City center → City outskirts → Suburban → Highway → Rural

3GPP-calibrated path loss, shadowing, shared bandwidth and queue delay create changing network conditions.

The live dashboard simulates the placement loop; it does not stream camera inference to real servers.

Three limits, then an edge-first choice

A tier must pass all feasibility checks before the policy compares it with other eligible tiers.

≤ 100 ms
End-to-end latency

≤ 15%
Accuracy loss

< 95%
Server utilization

1 Prefer the nearby edge

If both remote tiers pass, choose MEC when the cloud gains less than 5 mAP points. Here the gain is 2.7.

2 Use cloud when the edge fails

Cloud is selected only while it still meets latency, accuracy and capacity limits.

3 Fall back to the on-board GPU

If no tier passes, choose B: 5 ms inference, but 20.8% accuracy loss. This is a degraded decision.

Optional risk and stability controls

q90 gating adds a cautious delay estimate; dwell filtering reduces repeated switches. Defaults: mean gating, dwell off.

What the paper validates

Results below come from the repository experiments on generated data, rather than road trials.

PHYSICS

Within 1.6%

Configured physical relationships can be recovered from the generated dataset.

Checks internal consistency of the generator.

PREDICTION

6.12 ms MEC delay error

The compact HistGB predictor has 1.27 MB model size and 3.79 ms median prediction time.

Held-out synthetic samples.

PLACEMENT

94.0% → 96.6%

Latency compliance improves with q90 gating in the urban-sparse scenario.

EMA-smoothed policy, 1,000 frames, 10 Hz.

What the audience should take away

The framework makes the trade-offs visible and repeatable, including the moments when full QoS cannot be met.

Sources: new_paper/main.tex; experiments/results_exp2b.json; experiments/results_exp4.json.

Live demo: change the network and watch the decision

Run: python demo.py --outage-at 0 | Default scenario: suburban | Press q to finish

START

MEC / C

Normal connection: the edge gives sufficient accuracy with lower network cost.

O

CLOUD / D

Cut only the MEC link. Cloud takes over if it remains within every budget.

X

LOCAL GPU / B

Cut the remote options. The local model keeps running with an accuracy-budget breach.



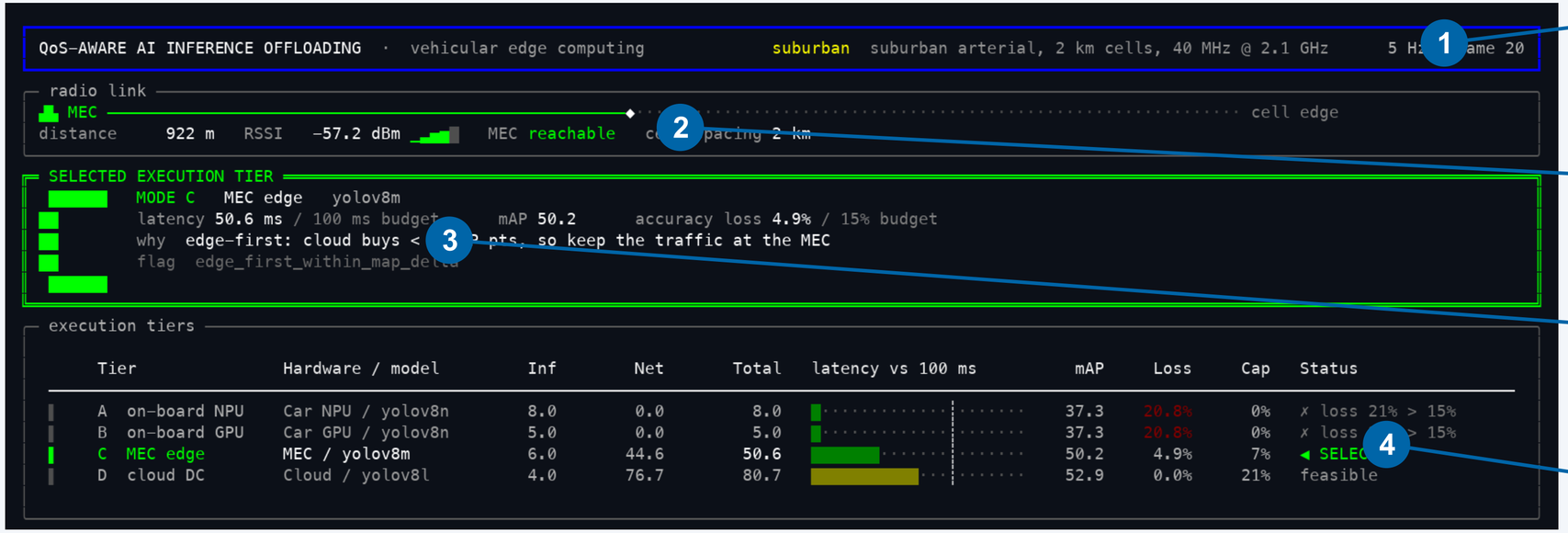
Faults last about 3 seconds at the current update rate, then the policy re-evaluates recovery.

1–5: road scenario space: pause + / −: speed c: cloud congestion r: reset k: kiosk

Blackout effects build through delay smoothing; a return to MEC depends on current conditions.

Dashboard walkthrough • radio, decision and tier comparison

Presenter reference 1/2 — an actual rendered dashboard state after 20 simulated frames.



1 Header

Road profile, radio parameters, Hz and frame count. PAUSED, AUTO and q90 appear when enabled.

2 Radio link

Diamond = vehicle. Distance = MEC separation; RSSI / bars = signal strength. Reachability and red fault labels show link status.

3 Selected tier

Large letter = destination. Latency, mAP and loss describe that tier. “why” explains it; “flag” is the reason code.

4 Execution table

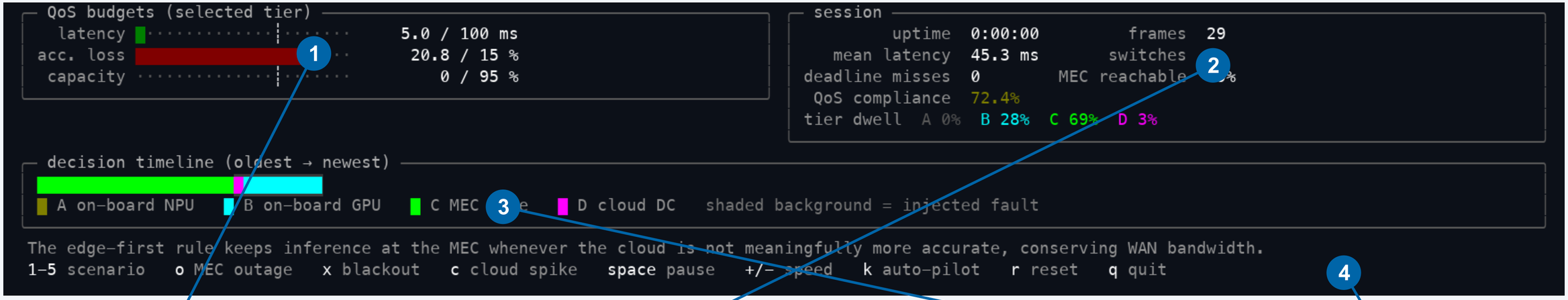
A/B/C/D compare candidates. The marker identifies the active tier; status shows eligibility or the first failed check.

Every table column

Inf = inference ms · Net = smoothed predicted network ms · Total = Inf + Net · mAP = detection score (higher is better).
Loss = average relative class-AP drop from YOLOv8l · Cap = predicted utilization · — = unavailable · bar marker = 100 ms.

Dashboard walkthrough • budgets, session and controls

Presenter reference 2/2 — a captured blackout state, showing the cost of the on-board fallback.



1 Selected-tier gauges

Latency, loss and utilization against their limits. Red means exceeded; the light marker shows the threshold. Here loss is 20.8% > 15%.

2 Session statistics

Uptime and frame count; average chosen-tier latency; tier switches; deadline misses; MEC-reachable frame share. Tier dwell = percentage of frames in each tier.

3 Timeline + compliance

Timeline runs oldest → newest. Color = tier; shaded background = injected fault. Compliance counts latency / accuracy breaches; capacity is checked separately by the policy.

4 Footer 1–5 = dense city / outskirts / suburban / highway / rural · o = MEC outage · x = blackout · c = cloud spike
space = pause · +/− = Hz · k = automatic scenario/fault cycle · r = reset counters/history · q = quit. Rotating text = tips.